Workshop 3

Maria Lucía Castillo García

Índice de entregables

Evidencia.....2

Parte 1: PostgreSQL.....2

Parte 2: Flask + PostgreSQL.....3

Parte 3: Variables de entorno.....4

Parte 4: Tres APIs.....5

Parte 5: Prueba de integración.....6

Parte 6: Tolerancia ante el fallo.....7

Reto - Parte 7.....8

Respuestas análisis.....12

Evidencia

```
casti@MLC MINGW64 ~/workshop2-docker
$ cd ~

casti@MLC MINGW64 ~
$ mkdir -p workshop3-docker/api workshop3-docker/db/init workshop3-docker/nginx

casti@MLC MINGW64 ~
$ cd workshop3-docker

casti@MLC MINGW64 ~/workshop3-docker
$ ls -R
.:
api/  db/  nginx/

./api:

./db:
init/

./db/init:

./nginx:
```

Parte 1: PostgreSQL

```
casti@MLC MINGW64 ~/workshop3-docker
$ touch db/init/01-init.sql

casti@MLC MINGW64 ~/workshop3-docker
$ notepad db/init/01-init.sql
```

```
CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    price NUMERIC(10,2) NOT NULL
);

INSERT INTO products (name, price) VALUES
('Laptop', 3500000),
('Mouse', 80000),
('Teclado', 150000),
('Monitor', 900000),
('Audifonos', 220000);
```

Parte 2: Flask + PostgreSQL

```
casti@MLC MINGW64 ~/workshop3-docker
$ touch api/app.py

casti@MLC MINGW64 ~/workshop3-docker
$ notepad api/app.py
```

```python
import os
import psycopg
from flask import Flask, jsonify

app = Flask(__name__)
INSTANCE = os.getenv("INSTANCE", "api-local")

def get_connection():
    return psycopg.connect(
        host=os.getenv("DB_HOST"),
        dbname=os.getenv("DB_NAME"),
        user=os.getenv("DB_USER"),
        password=os.getenv("DB_PASSWORD")
    )

@app.get("/products")
def products():
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT id, name, price FROM products ORDER BY id")
    rows = cur.fetchall()
    cur.close()
    conn.close()

    lista = []
    for row in rows:
        lista.append({
            "id": row[0],
            "name": row[1],
            "price": float(row[2])
        })

    return jsonify({
        "served_by": INSTANCE,
        "products": lista
    })

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

```
casti@MLC MINGW64 ~/workshop3-docker
$ touch api/requirements.txt

casti@MLC MINGW64 ~/workshop3-docker
$ notepad api/requirements.txt
```

```
Flask
psycopg[binary]
```

```
casti@MLC MINGW64 ~/workshop3-docker
$ touch api/Dockerfile

casti@MLC MINGW64 ~/workshop3-docker
$ notepad api/Dockerfile
```

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]
```

Parte 3: Variables de entorno

```
casti@MLC MINGW64 ~/workshop3-docker
$ touch docker-compose.yml

casti@MLC MINGW64 ~/workshop3-docker
$ notepad docker-compose.yml
```

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: shop
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    volumes:
      - ./db/init:/docker-entrypoint-initdb.d:ro

  api1:
    build: ./api
    environment:
      INSTANCE: api-1
      DB_HOST: db
      DB_NAME: shop
      DB_USER: postgres
      DB_PASSWORD: postgres
    depends_on:
      - db

  api2:
    build: ./api
    environment:
      INSTANCE: api-2
      DB_HOST: db
      DB_NAME: shop
      DB_USER: postgres
      DB_PASSWORD: postgres
    depends_on:
      - db

  api3:
    build: ./api
    environment:
      INSTANCE: api-3
      DB_HOST: db
      DB_NAME: shop
      DB_USER: postgres
      DB_PASSWORD: postgres
    depends_on:
      - db
```

```
nginx:
  image: nginx:alpine
  ports:
    - "8080:80"
  volumes:
    - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
  depends_on:
    - api1
    - api2
    - api3
```

Parte 4: Tres APIs

```
casti@MLC MINGW64 ~/workshop3-docker
$ touch nginx/nginx.conf

casti@MLC MINGW64 ~/workshop3-docker
$ notepad nginx/nginx.conf
```

```
worker_processes 1;

events {
    worker_connections 1024;
}

http {
    upstream flask_backend {
        server api1:5000;
        server api2:5000;
        server api3:5000;
    }

    server {
        listen 80;

        location / {
            proxy_pass http://flask_backend;
        }
    }
}
```

Parte 5: Prueba de integración

```
casti@MLC MINGW64 ~/workshop3-docker
$ docker compose up -d --build
[+] up 9/15
 - Image postgres:16 [ ##   ##### ## #] Pulling
```

```
[+] up 25/25esolving provenance for metadata file
 ✔ Image postgres:16                 Pulled
 ✔ Image workshop3-docker-api1       Built
 ✔ Image workshop3-docker-api2       Built
 ✔ Image workshop3-docker-api3       Built
 ✔ Network workshop3-docker_default  Created
 ✔ Container workshop3-docker-db-1   Started
 ✔ Container workshop3-docker-api1-1 Started
 ✔ Container workshop3-docker-api3-1 Started
 ✔ Container workshop3-docker-api2-1 Started
 ✔ Container workshop3-docker-nginx-1 Started

casti@MLC MINGW64 ~/workshop3-docker
$ docker compose ps
NAME                    IMAGE                   COMMAND                  SERVICE   CREATED         STATUS         PORTS
workshop3-docker-api1-1   workshop3-docker-api1   "python app.py"          api1      4 minutes ago   Up 4 minutes   5000/tcp
workshop3-docker-api2-1   workshop3-docker-api2   "python app.py"          api2      4 minutes ago   Up 4 minutes   5000/tcp
workshop3-docker-api3-1   workshop3-docker-api3   "python app.py"          api3      4 minutes ago   Up 4 minutes   5000/tcp
workshop3-docker-db-1     postgres:16             "docker-entrypoint.s…"   db        4 minutes ago   Up 4 minutes   5432/tcp
workshop3-docker-nginx-1  nginx:alpine           "/docker-entrypoint.…"   nginx     4 minutes ago   Up 4 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

```
casti@MLC MINGW64 ~/workshop3-docker
$ docker compose exec db psql -U postgres -d shop -c "SELECT * FROM products;"
 id |   name    |   price
----+-----------+------------
  1 | Laptop    | 3500000.00
  2 | Mouse     |   80000.00
  3 | Teclado   |  150000.00
  4 | Monitor   |  900000.00
  5 | Audifonos |  220000.00
(5 rows)
```

```
casti@MLC MINGW64 ~/workshop3-docker
$ curl http://localhost:8080/products
{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-1"}

casti@MLC MINGW64 ~/workshop3-docker
$ for i in {1..9}; do curl -s http://localhost:8080/products; echo; done
{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-2"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-3"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-1"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-2"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-3"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-1"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-2"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-3"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-1"}
```

Parte 6: Tolerancia ante el fallo

```
casti@MLC MINGW64 ~/workshop3-docker
$ docker compose stop api2
[+] stop 1/1
 ✔ Container workshop3-docker-api2-1 Stopped

casti@MLC MINGW64 ~/workshop3-docker
$ docker compose ps
NAME                        IMAGE                    COMMAND                  SERVICE   CREATED          STATUS          PORTS
workshop3-docker-api1-1     workshop3-docker-api1    "python app.py"          api1      19 minutes ago   Up 19 minutes   5000/tcp
workshop3-docker-api3-1     workshop3-docker-api3    "python app.py"          api3      19 minutes ago   Up 19 minutes   5000/tcp
workshop3-docker-db-1       postgres:16              "docker-entrypoint.s…"   db        19 minutes ago   Up 19 minutes   5432/tcp
workshop3-docker-nginx-1    nginx:alpine             "/docker-entrypoint.…"   nginx     19 minutes ago   Up 19 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

```
casti@MLC MINGW64 ~/workshop3-docker
$ for i in {1..9}; do curl -s http://localhost:8080/products; echo; done
{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-3"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-3"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-1"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-3"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-1"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-3"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-1"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-3"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-1"}
```

```
casti@MLC MINGW64 ~/workshop3-docker
$ docker compose stop api3
[+] stop 1/1
 ✔ Container workshop3-docker-api3-1 Stopped                                                                                                                                                                10.2s

casti@MLC MINGW64 ~/workshop3-docker
$ for i in {1..9}; do curl -s http://localhost:8080/products; echo; done
{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-1"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-1"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-1"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-1"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-1"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-1"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-1"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-1"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","
price":220000.0}],"served_by":"api-1"}
```

```
casti@MLC MINGW64 ~/workshop3-docker
$ docker compose stop api1
[+] stop 1/1
 ✔ Container workshop3-docker-api1-1 Stopped

casti@MLC MINGW64 ~/workshop3-docker
$ curl http://localhost:8080/products
<html>
<head><title>502 Bad Gateway</title></head>
<body>
<center><h1>502 Bad Gateway</h1></center>
<hr><center>nginx/1.31.4</center>
</body>
</html>
```

Restaurar de nuevo

```
casti@MLC MINGW64 ~/workshop3-docker
$ docker compose start api1 api2 api3
[+] start 3/3
 ✔ Container workshop3-docker-api2-1 Started
 ✔ Container workshop3-docker-api3-1 Started
 ✔ Container workshop3-docker-api1-1 Started

casti@MLC MINGW64 ~/workshop3-docker
$ docker compose ps
NAME                        IMAGE                       COMMAND                  SERVICE   CREATED          STATUS          PORTS
workshop3-docker-api1-1     workshop3-docker-api1       "python app.py"          api1      23 minutes ago   Up 7 seconds    5000/
workshop3-docker-api1-1     workshop3-docker-api1       "python app.py"          api1      23 minutes ago   Up 7 seconds    5000/tcp
workshop3-docker-api2-1     workshop3-docker-api2       "python app.py"          api2      23 minutes ago   Up 7 seconds    5000/tcp
workshop3-docker-api3-1     workshop3-docker-api3       "python app.py"          api3      23 minutes ago   Up 7 seconds    5000/tcp
workshop3-docker-db-1       postgres:16                 "docker-entrypoint.s…"   db        23 minutes ago   Up 23 minutes   5432/tcp
workshop3-docker-nginx-1    nginx:alpine                "/docker-entrypoint.…"   nginx     23 minutes ago   Up 23 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

```
casti@MLC MINGW64 ~/workshop3-docker
$ for i in {1..9}; do curl -s http://localhost:8080/products; echo; done
{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":220000.0}],"served_by":"api-1"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":220000.0}],"served_by":"api-1"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":220000.0}],"served_by":"api-2"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":220000.0}],"served_by":"api-3"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":220000.0}],"served_by":"api-1"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":220000.0}],"served_by":"api-2"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":220000.0}],"served_by":"api-3"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":220000.0}],"served_by":"api-1"}

{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":220000.0}],"served_by":"api-2"}
```

Reto - Parte 7

```
casti@MLC MINGW64 ~/workshop3-docker
$ notepad api/app.py
```

```python
import os
import psycopg
from flask import Flask, jsonify, request

app = Flask(__name__)
INSTANCE = os.getenv("INSTANCE", "api-local")

def get_connection():
    return psycopg.connect(
        host=os.getenv("DB_HOST"),
        dbname=os.getenv("DB_NAME"),
        user=os.getenv("DB_USER"),
        password=os.getenv("DB_PASSWORD")
    )

@app.get("/products")
def list_products():
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT id, name, price FROM products ORDER BY id")
    rows = cur.fetchall()
    cur.close()
    conn.close()

    lista = []
    for row in rows:
        lista.append({
            "id": row[0],
            "name": row[1],
            "price": float(row[2])
        })

    return jsonify({
        "served_by": INSTANCE,
        "products": lista
    })
```

```python
@app.get("/products/<int:id>")
def get_product(id):
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("SELECT id, name, price FROM products WHERE id = %s", (id,))
    row = cur.fetchone()
    cur.close()
    conn.close()

    if row is None:
        return jsonify({
            "served_by": INSTANCE,
            "error": "Producto no encontrado"
        }), 404

    return jsonify({
        "served_by": INSTANCE,
        "product": {
            "id": row[0],
            "name": row[1],
            "price": float(row[2])
        }
    })

@app.post("/products")
def create_product():
    data = request.get_json()

    conn = get_connection()
    cur = conn.cursor()
    cur.execute(
        "INSERT INTO products (name, price) VALUES (%s, %s) RETURNING id",
        (data["name"], data["price"])
    )
    nuevo_id = cur.fetchone()[0]
    conn.commit()
    cur.close()
    conn.close()

    return jsonify({
        "served_by": INSTANCE,
        "created_id": nuevo_id
    }), 201
```

```python
@app.put("/products/<int:id>")
def update_product(id):
    data = request.get_json()

    conn = get_connection()
    cur = conn.cursor()
    cur.execute(
        "UPDATE products SET name = %s, price = %s WHERE id = %s",
        (data["name"], data["price"], id)
    )
    filas = cur.rowcount
    conn.commit()
    cur.close()
    conn.close()

    if filas == 0:
        return jsonify({
            "served_by": INSTANCE,
            "error": "Producto no encontrado"
        }), 404

    return jsonify({
        "served_by": INSTANCE,
        "updated_id": id
    })

@app.delete("/products/<int:id>")
def delete_product(id):
    conn = get_connection()
    cur = conn.cursor()
    cur.execute("DELETE FROM products WHERE id = %s", (id,))
    filas = cur.rowcount
    conn.commit()
    cur.close()
    conn.close()

    if filas == 0:
        return jsonify({
            "served_by": INSTANCE,
            "error": "Producto no encontrado"
        }), 404

    return jsonify({
        "served_by": INSTANCE,
        "deleted_id": id
    })
```

```python
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

PRUEBAS DEL CRUD

```
casti@MLC MINGW64 ~/workshop3-docker
$ docker compose up -d --build
#1 [internal] load local bake definitions
#1 reading from stdin 1.45kB 0.0s done
#1 DONE 0.0s

#2 [api2 internal] load build definition from Dockerfile
#2 transferring dockerfile: 215B 0.0s done
#2 DONE 0.0s

#3 [api2 internal] load metadata for docker.io/library/python:3.12-slim
#3 DONE 1.8s

#4 [api1 internal] load .dockerignore
#4 transferring context: 2B done
#4 DONE 0.0s
```

...

```
#16 [api3] resolving provenance for metadata file
#16 DONE 0.0s
[+] up 8/8
 ✔ Image workshop3-docker-api3        Built
 ✔ Image workshop3-docker-api1        Built
 ✔ Image workshop3-docker-api2        Built
 ✔ Container workshop3-docker-nginx-1 Running
 ✔ Container workshop3-docker-db-1    Running
 ✔ Container workshop3-docker-api3-1  Started
 ✔ Container workshop3-docker-api2-1  Started
 ✔ Container workshop3-docker-api1-1  Started

casti@MLC MINGW64 ~/workshop3-docker
$ docker compose ps
NAME                      IMAGE                     COMMAND                  SERVICE   CREATED          STATUS          PORTS
workshop3-docker-api1-1   workshop3-docker-api1     "python app.py"          api1      16 seconds ago   Up 5 seconds    5000/tcp
workshop3-docker-api2-1   workshop3-docker-api2     "python app.py"          api2      16 seconds ago   Up 5 seconds    5000/tcp
workshop3-docker-api3-1   workshop3-docker-api3     "python app.py"          api3      16 seconds ago   Up 5 seconds    5000/tcp
workshop3-docker-db-1     postgres:16               "docker-entrypoint.s…"   db        6 hours ago      Up 6 hours      5432/tcp
workshop3-docker-nginx-1  nginx:alpine             "/docker-entrypoint.…"   nginx     6 hours ago      Up 6 hours      0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

PRUEBAS DEL CRUD

Estado Inicial

```
casti@MLC MINGW64 ~/workshop3-docker
$ curl -s http://localhost:8080/products; echo
{"products":[{"id":1,"name":"Laptop","price":3500000.0},{"id":2,"name":"Mouse","price":80000.0},{"id":3,"name":"Teclado","price":150000.0},{"id":4,"name":"Monitor","price":900000.0},{"id":5,"name":"Audifonos","price":220000.0}],"served_by":"api-2"}
```

POST – Crear

```
casti@MLC MINGW64 ~/workshop3-docker
$ curl -s -X POST http://localhost:8080/products -H "Content-Type: application/json" -d '{"name":"Webcam","price":320000}'; echo
{"created_id":6,"served_by":"api-1"}
```

```
casti@MLC MINGW64 ~/workshop3-docker
$ for i in {1..3}; do curl -s http://localhost:8080/products/6; echo; done
{"product":{"id":6,"name":"Webcam","price":320000.0},"served_by":"api-3"}

{"product":{"id":6,"name":"Webcam","price":320000.0},"served_by":"api-2"}

{"product":{"id":6,"name":"Webcam","price":320000.0},"served_by":"api-1"}
```

PUT- Actualizar

```
casti@MLC MINGW64 ~/workshop3-docker
$ curl -s -X PUT http://localhost:8080/products/6 -H "Content-Type: application/json" -d '{"name":"Webcam HD","price":450000}'; echo
{"served_by":"api-3","updated_id":6}


casti@MLC MINGW64 ~/workshop3-docker
$ curl -s http://localhost:8080/products/6; echo
{"product":{"id":6,"name":"Webcam HD","price":450000.0},"served_by":"api-2"}
```

DELETE – Eliminar

```
casti@MLC MINGW64 ~/workshop3-docker
$ curl -s -X DELETE http://localhost:8080/products/6; echo
{"deleted_id":6,"served_by":"api-1"}


casti@MLC MINGW64 ~/workshop3-docker
$ curl -s http://localhost:8080/products/6; echo
{"error":"Producto no encontrado","served_by":"api-3"}


casti@MLC MINGW64 ~/workshop3-docker
$ docker compose exec db psql -U postgres -d shop -c "SELECT * FROM products;"
 id |   name    |   price
----+-----------+------------
  1 | Laptop    | 3500000.00
  2 | Mouse     |   80000.00
  3 | Teclado   |  150000.00
  4 | Monitor   |  900000.00
  5 | Audifonos |  220000.00
(5 rows)
```

```
casti@MLC MINGW64 ~/workshop3-docker
$ docker compose down
[+] down 6/6
 ✔ Container workshop3-docker-nginx-1 Removed
 ✔ Container workshop3-docker-api3-1  Removed
 ✔ Container workshop3-docker-api2-1  Removed
 ✔ Container workshop3-docker-api1-1  Removed
 ✔ Container workshop3-docker-db-1    Removed
 ✔ Network workshop3-docker_default   Removed
```

Respuestas análisis

1. ¿Dónde se encuentra el estado del negocio?

El estado del negocio se encuentra en PostgreSQL. Las APIs no almacenan información propia; simplemente consultan la base de datos cada vez que reciben una petición.

Por eso, aunque las respuestas provenían de distintas instancias (api1, api2 o api3), todas devolvían exactamente los mismos productos: la información está centralizada en una única base de datos.

2. ¿Por qué las APIs pueden ser stateless?

Pueden ser stateless porque no guardan información entre peticiones. Todo lo que necesitan para responder lo reciben en la solicitud o lo consultan directamente en la base de datos.

Esto hace que las tres instancias sean equivalentes: cualquiera puede atender cualquier petición. Gracias a ello es posible balancear la carga y agregar más instancias cuando se necesita mayor capacidad.

3. ¿Qué ocurre si una API reinicia?

Prácticamente nada desde la perspectiva del usuario. Mientras la instancia está fuera de servicio, Nginx envía las peticiones a las demás APIs disponibles.

Cuando la instancia vuelve a levantarse, se reintegra automáticamente y continúa trabajando con la misma base de datos que las demás.

4. ¿Qué ocurre si PostgreSQL reinicia?

Si PostgreSQL deja de estar disponible, las tres APIs dejan de funcionar porque todas dependen de ella para obtener la información.

Si la base de datos simplemente se reinicia, el servicio vuelve a funcionar cuando termina el proceso. Sin embargo, en esta práctica los datos están almacenados dentro del contenedor, por lo que si este se elimina y se crea nuevamente, la información se pierde. Para evitarlo se deben utilizar volúmenes persistentes.

5. ¿Dónde existen puntos únicos de falla?

Existen principalmente dos:

El primero es Nginx, ya que todas las peticiones pasan por él. Si falla, los usuarios no pueden acceder al sistema.

El segundo es PostgreSQL, que además de ser única almacena toda la información. Si la base de datos falla, ninguna API puede responder correctamente.

6. ¿El balanceamiento de API soluciona el escalamiento de la base de datos?

No. El balanceamiento ayuda a repartir las peticiones entre varias APIs, pero todas siguen consultando la misma base de datos.

De hecho, al agregar más APIs también aumenta la cantidad de consultas hacia PostgreSQL. Por eso, aunque las APIs escalen correctamente, la base de datos puede convertirse en el nuevo cuello de botella.

7. ¿Qué mecanismos podrían añadirse en una arquitectura de producción?

Para una arquitectura real se podrían agregar varias mejoras:

- **Persistencia:** utilizar volúmenes (espacios de almacenamiento independientes del contenedor) para guardar los datos y realizar copias de seguridad periódicas, de modo que la información no se pierda si el contenedor de PostgreSQL se elimina o se vuelve a crear.
- **Disponibilidad del punto de entrada**: más de una instancia de Nginx detrás de un balanceador externo, para dejar de ser punto único de falla.
- **Salud del sistema:** health checks activos que saquen del balanceo la instancia enferma antes de que un usuario reciba el error, más monitoreo y registro centralizado.
- **Seguridad:** las credenciales fuera del docker-compose.yml, gestionadas como secretos.